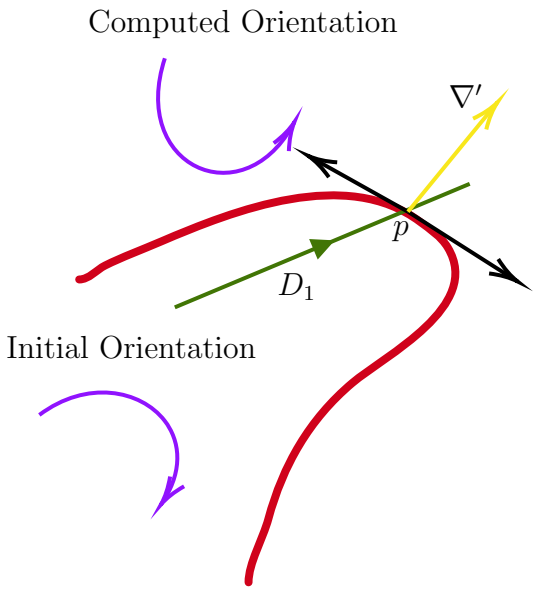

Figure 2.3: A sample execution of GRADIENTDIRECTION. After computing the cross product between the direction of D_1 (arrowed green line) and ∇' (arrowed yellow line), the algorithm compares the orientation (counter clockwise) with the initial orientation of the drones (clockwise), and decides to go clockwise from ∇'.

Algorithm 1 Initially, robots are $\sqrt{\lambda}$ apart; one inside and one outside

```
1: procedure BOUNDARY-SKETCH(λ)                                   ▷
2: D1, D2 ← the two robots
3: ∇ ← boundary gradient at point of crossing with line segment between D1 and
   D2
4: α ← √λ
5:     while Incomplete(D1,D2) do
6:         if inside (D1) XOR inside (D2) then
7:             D1 and D2 both move λ distance in the direction of ∇
8:         end if
9:         if inside (D1) = false and inside (D2) = false then
10:            α ← √λ
11:            CROSS-BOUNDARY(D1, D2, α)
12:        elseif inside (D1) = true and inside (D2) = true
13:            α ← −√λ
14:            CROSS-BOUNDARY (D2, D1, α)
15:        end if
16:    end while
17: end procedure
```

Algorithm 2 Finds the direction of the gradient at a given point p of intersection

procedure GRADIENTDIRECTION(p, D_1, D_2)
  Orient $\leftarrow$ inside (D_1) XOR inside (D_2)
  Initial Orientation $\leftarrow$ **false**
  $\nabla' \leftarrow$ Estimated normal vector to the gradient at p
  Sign $\leftarrow$ CrossProductSign (Direction vector of D_1, ∇').
  if Orient = Initial Orientation **then**
    Orient $\leftarrow$ inside (D_1)
  end if
  if Orient = **Sign then**
    return $\nabla' + \pi/2$
  else
    return $\nabla' - \pi/2$
  end if
end procedure

Algorithm 3 Reestablishes "Sandwich" Invariant

procedure CROSS-BOUNDARY(D_1, D_2, α)
  $p \leftarrow$ last position of D_1 before crossing
  $R \leftarrow$ the vertices of the regular polygon including D_1's position with exterior angle $\sqrt{\lambda}$ and the edge beginning at D_1's position facing the direction of $\nabla + \alpha$.
  $P \leftarrow$ the vertices of the convex hull of $R \cup \{p\}$. For all $i : 0 \leq i \leq |P| - 1$, let P_i be the i-th vertex in this convex hull, ordered such that $P_0 = p$ and $P_1 = D_1$'s current position.
  $\nabla \leftarrow$ gradient at the last boundary crossing of D_1
  $i \leftarrow 1$.
  while neither robot has crossed the boundary AND $i + 1 < |P|$ **do**
    D_1 moves to P_{i+1}.
    D_2 moves to closest point from it that is $\sqrt{\lambda}$ distance away from P_i and orthogonal to $\nabla + i\alpha$
    $i \leftarrow i + 1$
  end while
  while neither robot has crossed the boundary **do**
    D_1 moves towards point p taking steps of length λ.
    D_2 moves to closest point from it that is $\sqrt{\lambda}$ distance away from D_1 and orthogonal to D_1's direction.
  end while
  if D_2 crossed the boundary **then**
    SYNCHRONIZE (D_1, D_2)
  else
    $\nabla \leftarrow$ the current direction of D_1.
  end if
end procedure

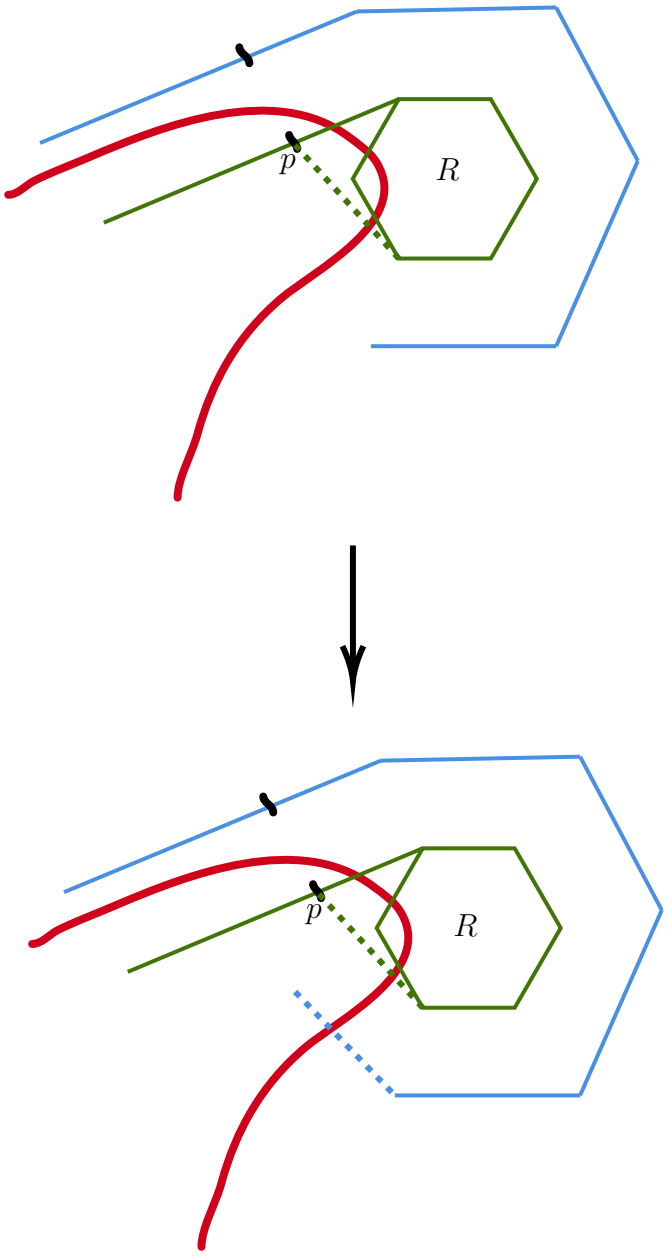

Figure 2.4: A high level execution of the CROSS-BOUNDARY subroutine, where the top part indicates the execution of the Algorithm from lines 1 through 10 and the bottom part the rest. Here a small regular polygon R is drawn and then a convex hull P is considered from the vertices of R and the starting position p of the robot moving across the green path. Upon crossing the boundary again, the subroutine SYNCHRONIZE is invoked (see Figure 2.5)

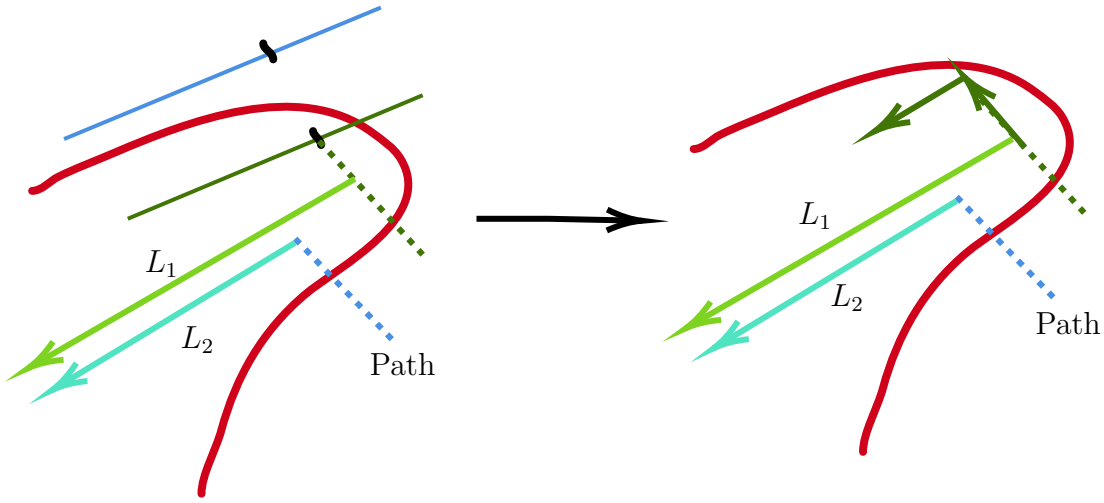

Figure 2.5: A sample execution of SYNCHRONIZE. Here dotted blue line indicates a segment of the Path of D_2, with L_1, L_2 as indicated in the algorithm.

Algorithm 4 Ensures the robots are at distance $\sqrt{\lambda}$ from each other and are oriented in the same direction.

procedure SYNCHRONIZE(D_1, D_2)
  Path $\leftarrow$ the polyline path of D_2 from last crossing of BOUNDARY-SKETCH with the shape till current position.
  $\nabla \leftarrow$ the gradient at the last boundary crossing for D_2.
  $L_1 \leftarrow$ the line in the direction of ∇ through D_1's position.
  $L_2 \leftarrow$ the line in the direction of ∇ through D_2's position.
  if L_1 crosses **Path then**
    Move D_2 in its current direction until it is $\sqrt{\lambda}$ distance away from L_1. Change direction to ∇ and take a single step of length λ.
    Move D_1 along L_1 until it is $\sqrt{\lambda}$ away from D_2.
  else
    Move D_1 in its current direction until it is $\sqrt{\lambda}$ distance away from L_2. Change direction to ∇ and move until the distance from D_2 is $\sqrt{\lambda}$.
    Swap (D_1, D_2).
  end if
end procedure

Algorithm 5 Initially, UASs D_1, D_2 are $\sqrt{\lambda}$ apart; one inside and one outside

```
1: procedure SKETCH(λ)                                                    ▷
2: ∇ ← boundary gradient; SketchTerminate ← false;
3:   while SketchTerminate = false do
4:     if inside (D_1) XOR inside (D_2) then
5:       Move λ distance in the direction of ∇
6:     end if
7:     if not inside (D_1) and not inside (D_2) then
8:       CROSS-BOUNDARY(D_1, D_2, −√λ)
9:     elseif inside (D_1) and inside (D_2)
10:      CROSS-BOUNDARY (D_2, D_1, √λ)
11:    end if
12:  end while
13: end procedure
```

ditionally, Euler et al. [20] describe an adaptive sampling strategy for efficient spatial mapping in large-scale environments, through cooperative UASs. Assenine et al. [4] developed a cooperative deep reinforcement learning approach that focuses on real-time monitoring of pollution plumes using a fleet of drones equipped with advanced sensing technology. Rossi and Brunelli [45] describe a team of UASs equipped with electronic noses for effective gas detection and mapping. Karbach et al. [34] use UAS to observe volcanic plume chemistry with ultralight sensor systems. Asadzadeh et al. review [3] the state-of-the-art in UAS-based remote sensing for the petroleum industry and environmental monitoring. Saldaña et al. [49] approximate boundaries of a 2D surface oil slick using aquatic robots taking pointwise measurements.

In comparison to these earlier results which make strong assumptions on the boundary shape (e.g. convexity or star-convexity [28, 31]), our

Algorithm 6 Initially, each pair of robots are $\sqrt{\lambda}$ apart; one clockwise and one counterclockwise to the *critical path*, referred as inside and outside respectively. They are near some critical point that is unknown but estimated to be close enough.

procedure GENERALIZED-SKETCH(λ) ▷
$D_1, D_2 \leftarrow$ the two pairs of robots
$\nabla \leftarrow$ boundary gradient at point of crossing with line segment between D_1 and D_2
$\alpha \leftarrow \sqrt{\lambda}$
    while Source not Found **do**
        if inside (D_1) XOR outside (D_2) **then**
            D_1 and D_2 both move λ distance in the direction of ∇
        end if
        if inside (D_1) = **false** and outside (D_2) = **false then**
            $\alpha \leftarrow \sqrt{\lambda}$
            CROSS-BOUNDARY(D_1, D_2, α)
        elseif inside (D_1) = **true** and outside (D_2) = **true**
            $\alpha \leftarrow -\sqrt{\lambda}$
            CROSS-BOUNDARY (D_2, D_1, α)
        end if
    end while
end procedure

Algorithm 7 A point p on a contour is given as input, and BOUNDARY-SKETCH is run to estimate the contour with value at p

procedure MULTIPLE-SOURCE(p) ▷
$P \leftarrow$ the set of critical points on the input contour.
Execute GENERALIZED-SKETCH starting near each point from the set P.
    while All Sources have not been Found AND $P \neq \phi$ **do**
        Execute every instance of GENERALIZED-SKETCH.
        if A source s has been found **then**
            MULTIPLE-SOURCE (s).
        end if
    end while
end procedure